TRƯỜNG ĐẠI HỌC
SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
HCMC University of Technology and Education
HCMUTE

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN CÔNG NGHỆ PHẦN MỀM

CÔNG NGHỆ PHẦN MỀM MỚI (MTSE431179)

Middleware

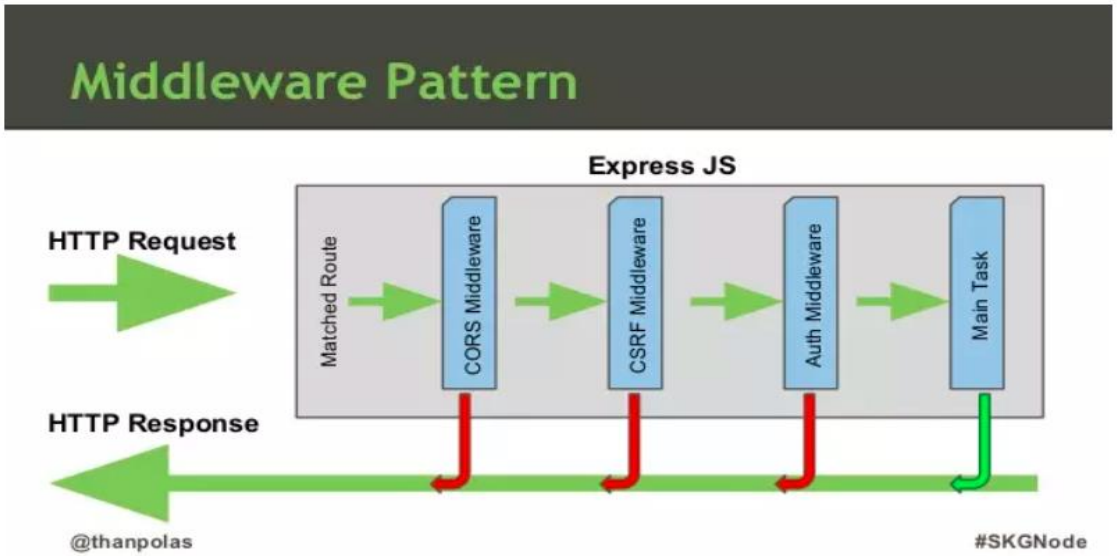
THS. NGUYỄN HỮU TRUNG

Thông tin Giảng viên

- Ths. Nguyễn Hữu Trung
- Khoa Công Nghệ Thông Tin
- Trường Đại học Sư Phạm Kỹ Thuật TP.HCM
- 090.861.7108
- trungnh@hcmute.edu.vn
- https://www.youtube.com/@baigiai

Định nghĩa về Middleware

- Middleware là cầu nối giữa tương tác của người dùng và phần nhân của hệ thống, trong lập trình Web, Middleware sẽ đóng vai trò trung gian giữa **request/response** (tương tác với người dùng) và các xử lý logic bên trong web server.
- Middleware trong các Framework lập tình Web (Django, Rails, ExpressJS), sẽ là các hàm được dùng để tiền xử lý, lọc các request trước khi đưa vào xử lý logic hoặc điều chỉnh các response trước khi gửi về cho người dùng.



Định nghĩa về Middleware

- Hình trên mô tả 3 middleware có trong ExpressJS. Một request khi gửi đến Express sẽ được xử lý qua 5 bước như sau :
  1. Tìm Route tương ứng với request
  2. Dùng CORS Middleware để kiểm tra cross-origin Resource sharing của request
  3. Dùng CRSF Middleware để xác thực CSRF của request, chống fake request
  4. Dùng Auth Middleware để xác thực request có được truy cập hay không
  5. Xử lý công việc được yêu cầu bởi request (Main Task)
- Bất kỳ bước nào trong các bước 2,3,4 nếu xảy ra lỗi sẽ trả về response thông báo cho người dùng, có thể là lỗi CORS, lỗi CSRF hay lỗi auth tùy thuộc vào request bị dừng ở bước nào.

Middleware trong ExpressJS

- ExpressJS khi hoạt động, về cơ bản sẽ là một loạt các hàm **Middleware** được thực hiện liên tiếp nhau. Sau khi đã thiết lập, các request từ phía người dùng khi gửi lên ExpressJS sẽ thực hiện lần lượt qua các hàm Middleware cho đến khi trả về response cho người dùng. Các hàm này sẽ được quyền truy cập đến các đối tượng đại diện cho Request - **req**, Response - **res**, hàm Middleware tiếp theo - **next**, và đối tượng lỗi - **err** nếu cần thiết.
- Một hàm Middleware sau khi hoạt động xong, nếu chưa phải là cuối cùng trong chuỗi các hàm cần thực hiện, sẽ cần gọi lệnh **next()** ể chuyển sang hàm tiếp theo, bằng không xử lý sẽ bị treo tại hàm đó.

Middleware trong ExpressJS

- Các chức năng mà middleware có thể thực hiện trong ExpressJS sẽ bao gồm :
  - Thực hiện bất cứ đoạn code nào
  - Thay đổi các đối tượng request và response
  - Kết thúc một quá trình request-response
  - Gọi hàm middleware tiếp theo trong stack
- Trong Express, có 5 kiểu middleware có thể sử dụng :
  - Application-level middleware (middleware cấp ứng dụng)
  - Router-level middleware (middlware cấp điều hướng - router)
  - Error-handling middleware (middleware xử lý lỗi)
  - Built-in middleware (middleware sẵn có)
  - Third-party middleware (middleware của bên thứ ba)

Application-level middleware

- Khi khởi tạo một Web Application với ExpressJS, chúng ta sẽ có một đối tượng đại diện cho Web App đó, thường được gán với biến app. Đối tượng này có thể khai báo các middleware thông qua các hàm : app.use() hoặc app.METHOD (trong đó METHOD sẽ là cá kiểu HTTP Method được ExpressJS hỗ trợ, dưới dạng tên là chữ viết thường, vd: app.get(), app.post()).
- Ví dụ dưới đây mô tả một hàm ko khai báo đường dẫn cụ thể, do đó nó sẽ được thực hiện mỗi lần request:

```
var app = express()

app.use(function (req, res, next) {
  console.log('Time:', Date.now())
  next()
})
```

Application-level middleware

- Ví dụ dưới đây dùng hàm use đến đường dẫn /user/:id. Hàm này sẽ được thực hiện mỗi khi request đến đường dẫn /user/:id bất kể phương thức nào (GET, POST,...):

```
app.use('/user/:id', function (req, res, next) {
  console.log('Request Type:', req.method)
  next()
})
```

- Tiếp theo là một ví dụ cho hàm được thực hiện mỗi khi truy cập đến đường dẫn /user/:id bằng phương thức GET:

```
app.get('/user/:id', function (req, res, next) {
  res.send('USER')
})
```

Application-level middleware

- Khi muốn gọi một loạt hàm middleware cho một đường dẫn cụ thể, chúng ta có thể thực hiện như ví dụ dưới đây, bằng cách khai báo liên tiếp các tham số là các hàm sau tham số đường dẫn :

```
app.use('/user/:id', function (req, res, next) {
  console.log('Request URL:', req.originalUrl)
  next()
}, function (req, res, next) {
  console.log('Request Type:', req.method)
  next()
})
```

- Hoặc chúng ta có thể tách ra thành 2 lần khai báo app.use, gọi là multiple routes, tuy nhiên ở các hàm phía trước cần gọi hàm next() khi kết thúc mỗi hàm, nếu không như ví dụ dưới đây, route thứ 2 sẽ không bao giờ được thực hiện do hàm thứ 2 trong route thứ nhất không gọi đến hàm next():

```
app.get('/user/:id', function (req, res, next) {
  console.log('ID:', req.params.id)
  next()
}, function (req, res, next) {
  res.send('User Info')
})

// handler for the /user/:id path, which prints the user ID
app.get('/user/:id', function (req, res, next) {
  res.end(req.params.id)
})
```

Application-level middleware

- Khi muốn bỏ qua các hàm middleware tiếp theo không thực hiện nữa, chúng ta sẽ sử dụng lệnh next('route'), tuy nhiên việc này chỉ tác dụng vưới các hàm middleware được load thông qua hàm app.METHOD hoặc router.METHOD.
- Ví dụ dưới đây mô tả một hàm middleware sẽ kết thúc ngạy lập tức khi tham số id=0:

```
app.get('/user/:id', function (req, res, next) {
  // if the user ID is 0, skip to the next route
  if (req.params.id === '0') next('route')
  // otherwise pass the control to the next middleware function in this stack
  else next()
}, function (req, res, next) {
  // render a regular page
  res.render('regular')
})

// handler for the /user/:id path, which renders a special page
app.get('/user/:id', function (req, res, next) {
  res.render('special')
})
```

Router-level middleware

- Các middleware này về chức năng không khác gì so với application-level middlewware ở trên, tuy nhiên thay vì dùng biến app có thể gây nhầm lẫn với các thiết lập, phần router có thể không rõ ràng và khó phân biệt, ExpressJS cung cấp một đối tượng router chuyên dùng để khai báo route bằng cách gọi hàm sau:

```
var router = express.Router()
```

- Phần code dưới đây mô tả một cách sử dụng router để thiết lập các route cần thiết cho một resource có tên là user:

```
var app = express()
var router = express.Router()

// a middleware function with no mount path. This code is executed for every reque
router.use(function (req, res, next) {
  console.log('Time:', Date.now())
  next()
})

// a middleware sub-stack shows request info for any type of HTTP request to the /
router.use('/user/:id', function (req, res, next) {
  console.log('Request URL:', req.originalUrl)
  next()
}, function (req, res, next) {
  console.log('Request Type:', req.method)
  next()
})

// a middleware sub-stack that handles GET requests to the /user/:id path
router.get('/user/:id', function (req, res, next) {
  // if the user ID is 0, skip to the next router
  if (req.params.id === '0') next('route')
  // otherwise pass control to the next middleware function in this stack
  else next()
}, function (req, res, next) {
  // render a regular page
  res.render('regular')
})
```

```
// handler for the /user/:id path, which renders a special page
router.get('/user/:id', function (req, res, next) {
  console.log(req.params.id)
  res.render('special')
})

// mount the router on the app
app.use('/', router)
```

Error-handling middleware

- Đây là các middleware phục vụ cho việc xử lý lỗi. Một lưu ý là các hàm cho việc này luôn nhận bốn tham số (*err, req, res, next*). Khi muốn khai báo một middlware cho việc xử lý lỗi, bạn cần tạo một hàm có 4 tham số đầu vào. Mặc dù bạn có thể không cần sử dụng đối tượng *next*, nhưng hàm vẫn cần format với bốn tham số như vậy. Nếu không ExpressJS sẽ không thể xác định đó là hàm xử lý lỗi, và sẽ không chạy khi có lỗi xảy ra, chỉ hoạt động giống như các hàm middlware khác.
- Đoạn code dưới đây mô tả một hàm xử lý lỗi truyền về cho client lỗi 500 khi có lỗi xảy ra từ server:

```
app.use(function (err, req, res, next) {
  console.error(err.stack)
  res.status(500).send('Something broke!')
})
```

Built-in middleware

- Kể từ phiên bản **4.x**, ExpressJS đã không còn phụ thuộc vào thư viện Connect. Ngoài middleware express.static, tất cả các hàm middleware khác đều đã được tách ra thành các modules riêng biệt. Điều này cung cấp cách tối ưu hóa và tùy chỉnh ứng dụng ExpressJS một cách linh hoạt nhất, giúp bạn tạo ra một ứng dụng Web Application phù hợp với nhu cầu, không bị thừa những thứ không cần thiết.
- Chỉ có một Built-in middlware duy nhất còn lại trong ExpressJS là express.static, dựa trên thư viện serve-static, được dùng để cung cấp các nội dung tĩnh trong trang Web, ví dụ như các trang HTML tĩnh, các file hình ảnh, css, js, ...
- Đoạn dưới đây mô tả việc sử dụng express.static để tạo ra một thư mục có tên là public, người dùng có thể truy cập các file html và htm trong thư mục này:

```
var options = {
  dotfiles: 'ignore',
  etag: false,
  extensions: ['htm', 'html'],
  index: false,
  maxAge: '1d',
  redirect: false,
  setHeaders: function (res, path, stat) {
    res.set('x-timestamp', Date.now())
  }
}

app.use(express.static('public', options))
```

```
app.use(express.static('public'))
app.use(express.static('uploads'))
app.use(express.static('files'))
```

Third-party middleware

- Sử dụng Third-party sẽ giúp chúng ta thêm các chức năng cho Web App của mình mà không cần mất nhiều công implement.
- Chúng ta sẽ cần cài đặt module thông qua npm, sau đó khai báo sử dụng trong đối tượng app nếu dùng ở Application-level, hoặc qua đối tượng router nếu dùng ở Router-level.
- Đoạn code sẽ cài đặt và sử dụng một middlware có tên là cookie-parser dùng để đọc cookies của request:

```
$ npm install cookie-parser
```

```
var express = require('express')
var app = express()
var cookieParser = require('cookie-parser')

// load the cookie-parsing middleware
app.use(cookieParser())
```

https://expressjs.com/en/resources/middleware.html